ARIMA Analysis of Kalshi Prediction Market Prices

Load and Prepare Data

```
trades_raw <- read_csv("trades_clean.csv")
trades_mutate <- mutate(trades_raw,
                        created_time = as.POSIXct(created_time, tz = "UTC"))
trades <- arrange(trades_mutate, ticker, created_time)

ticker_outcomes <- distinct(trades, ticker, outcome)
trade_counts_raw <- count(trades, ticker, sort = TRUE)
trade_counts <- left_join(trade_counts_raw, ticker_outcomes, by = "ticker")

cat("Total trades:", nrow(trades), "\n")
```

```
## Total trades: 106955
```

```
cat("Unique tickers:", n_distinct(trades$ticker), "\n")
```

```
## Unique tickers: 347
```

```
cat("Tickers with >= 20 trades:", sum(trade_counts$n >= 20), "\n")
```

```
## Tickers with >= 20 trades: 347
```

Single Contract Detailed Walkthrough

```
selected_ticker <- trade_counts$ticker[1]
cat("Selected ticker:", selected_ticker, "\n")
```

```
## Selected ticker: KXUSAIRANAGREEMENT-27-26APR
```

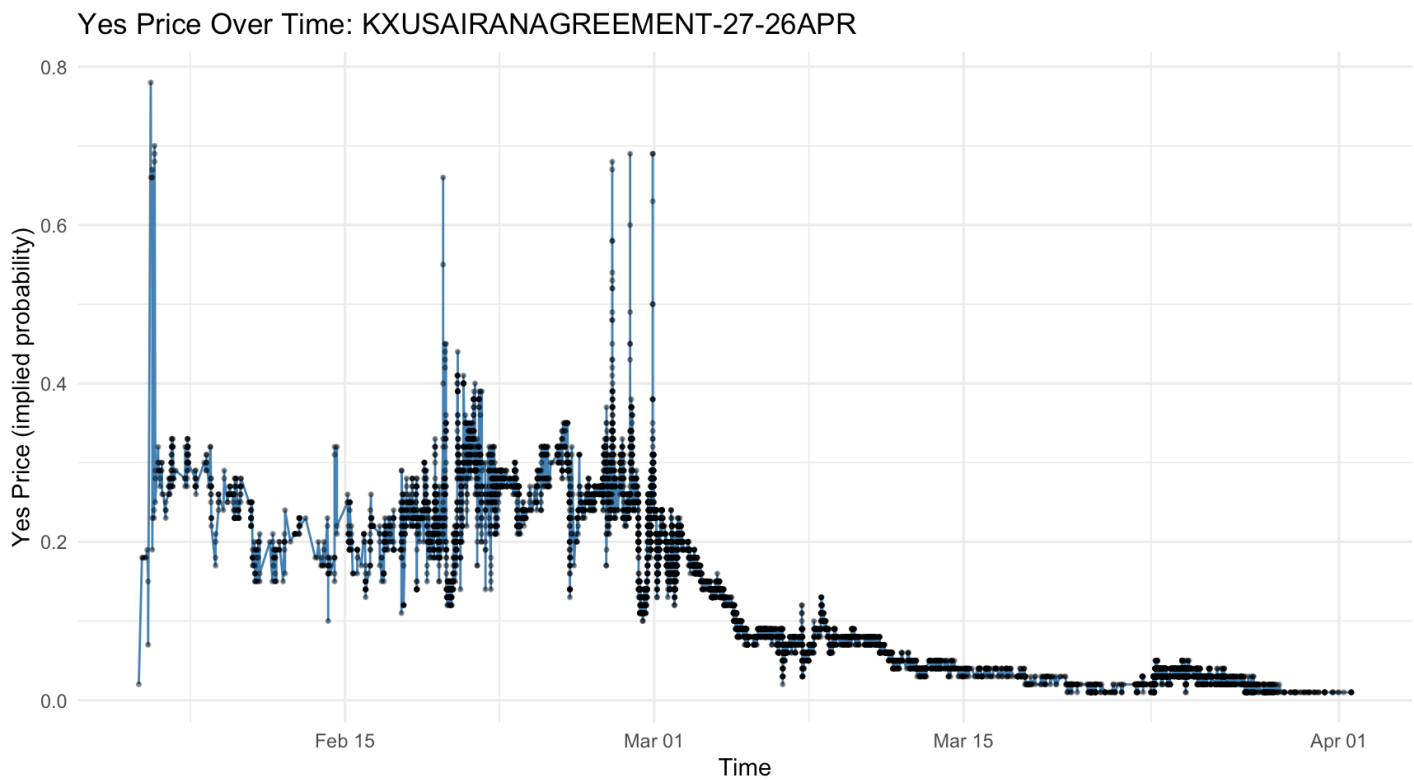
```
contract_trades_raw <- filter(trades, ticker == selected_ticker)
contract_trades_sel <- select(contract_trades_raw,
                              created_time, yes_price, outcome)
contract_trades <- arrange(contract_trades_sel, created_time)

cat("Number of trades:", nrow(contract_trades), "\n")
```

```
## Number of trades: 8379
```

```
cat("Outcome:", unique(contract_trades$outcome), "\n")
```

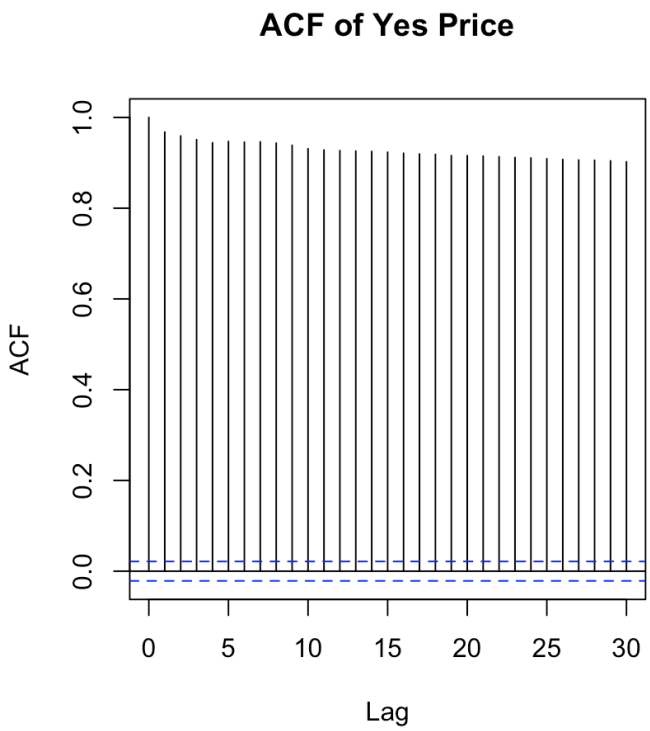
```
## Outcome: 0
```

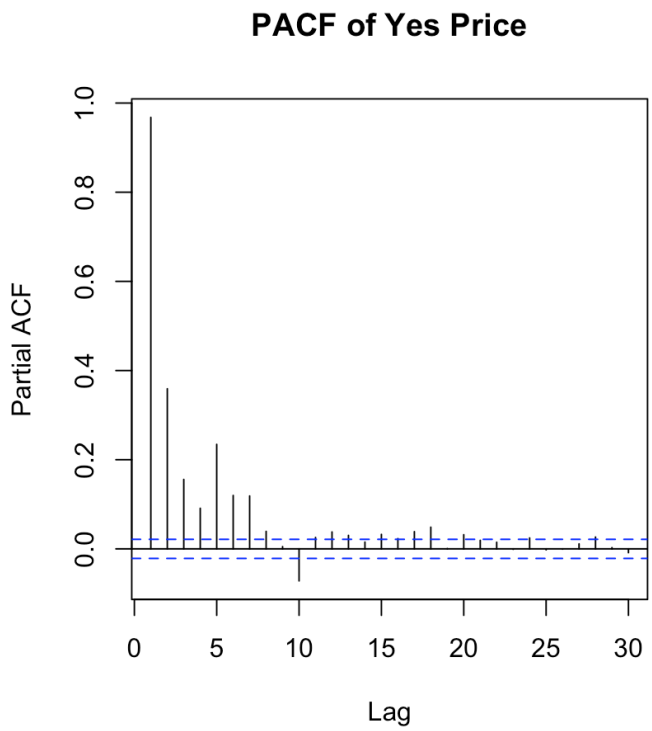
```
ggplot(contract_trades, aes(x = created_time, y = yes_price)) +
  geom_line(color = "steelblue") +
  geom_point(size = 0.6, alpha = 0.4) +
  labs(
    title = paste("Yes Price Over Time:", selected_ticker),
    x = "Time",
    y = "Yes Price (implied probability)"
  ) +
  theme_minimal()
```



ACF and PACF of Raw Price Series

```
price_ts <- ts(contract_trades$yes_price)

par(mfrow = c(1, 2))
acf(price_ts, main = "ACF of Yes Price", lag.max = 30)
pacf(price_ts, main = "PACF of Yes Price", lag.max = 30)
```






```
par(mfrow = c(1, 1))
```

Stationarity Test (ADF)

```
adf_result <- adf.test(price_ts)
print(adf_result)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  price_ts
## Dickey-Fuller = -5.2591, Lag order = 20, p-value = 0.01
## alternative hypothesis: stationary
```

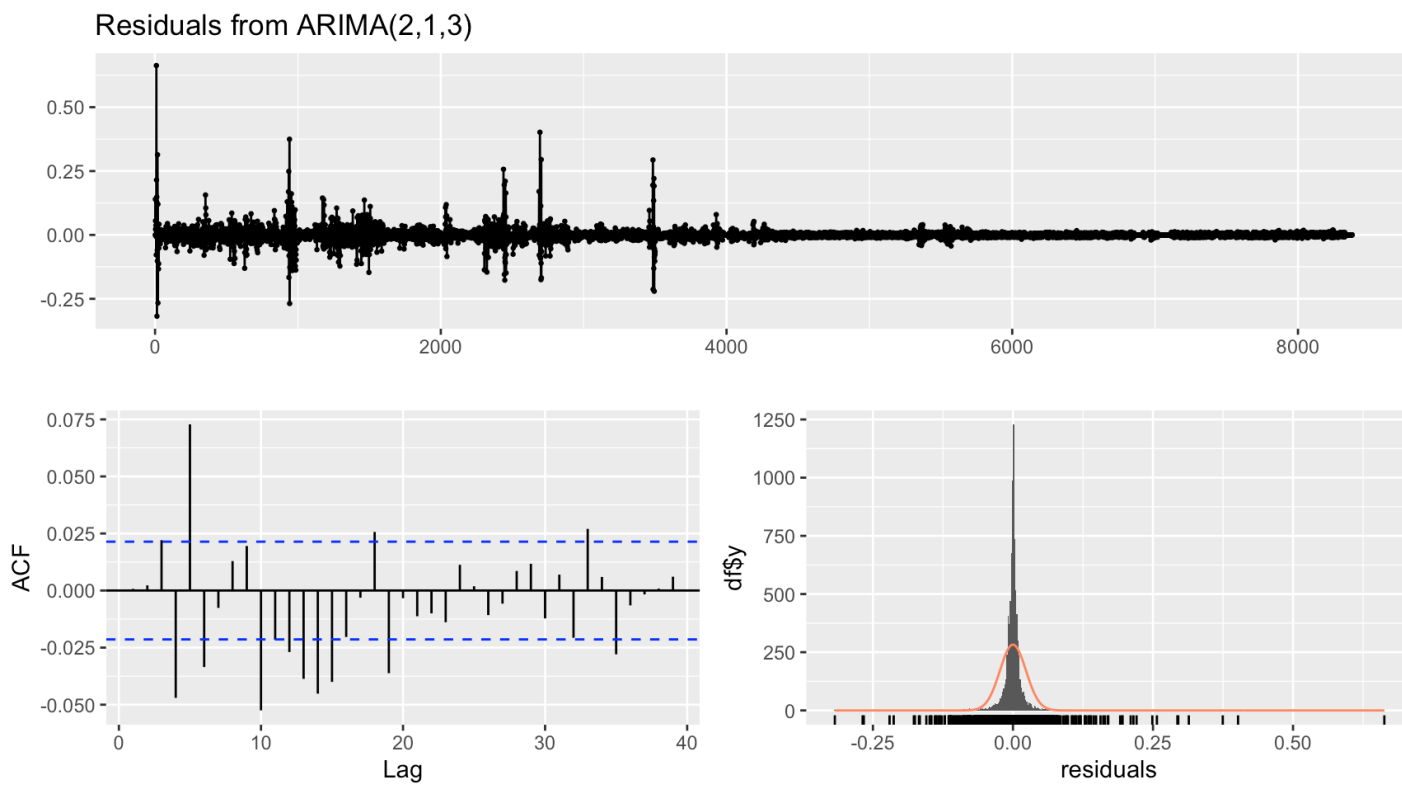
Fit ARIMA Model

```
arima_model <- auto.arima(price_ts, stepwise = FALSE, approximation = FALSE)
summary(arima_model)
```

```
## Series: price_ts
## ARIMA(2,1,3)
##
## Coefficients:
##          ar1      ar2      ma1     ma2      ma3
##       1.1288  -0.8126  -1.6560  1.3632  -0.4772
## s.e.  0.0334   0.0248   0.0336  0.0409   0.0165
##
## sigma^2 = 0.0005385:  log likelihood = 19643.92
## AIC=-39275.85   AICc=-39275.84   BIC=-39233.65
##
## Training set error measures:
##                        ME       RMSE       MAE       MPE     MAPE     MASE
## Training set -2.160986e-05 0.02319677 0.0101374 -1.671223 8.356032 1.048144
##                      ACF1
## Training set 0.0007628396
```

Residual Diagnostics

```
checkresiduals(arima_model)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(2,1,3)
## Q* = 104.81, df = 5, p-value < 2.2e-16
##
## Model df: 5.   Total lags used: 10
```

```
cat("ARIMA model order: ARIMA(",
    arima_model$arma[1], ",",
    arima_model$arma[6], ",",
    arima_model$arma[2], ")\n")
```

```
## ARIMA model order: ARIMA( 2 , 1 , 3 )
```

ARIMA Across All Tickers

Run auto.arima on every ticker with at least 20 trades

```
eligible_tickers <- pull(filter(trade_counts, n >= 20), ticker)

fit_arima_ticker <- function(tkr) {
  df <- arrange(filter(trades, ticker == tkr), created_time) # nolint

  ts_data <- ts(df$yes_price)
  n <- length(ts_data)
  n_train <- floor(n * 0.8)

  tryCatch({
    # fit on training portion
    train_ts <- ts(ts_data[1:n_train])
    model <- auto.arima(train_ts, stepwise = TRUE, approximation = TRUE)

    # forecast test portion
    n_test <- n - n_train
    fcast <- forecast(model, h = n_test)
    test_vals <- as.numeric(ts_data[(n_train + 1):n])
    rmse <- sqrt(mean((fcast$mean - test_vals)^2))
    mae <- mean(abs(fcast$mean - test_vals))

    # diagnostics on full-series model
    full_model <- auto.arima(ts_data, stepwise = TRUE, approximation = TRUE)
    lb <- Box.test(residuals(full_model),
                   lag = min(20, floor(n / 3)),
                   type = "Ljung-Box")
    adf <- adf.test(ts_data)

    data.frame(
      ticker = tkr,
      n_trades = n,
      outcome = as.integer(unique(df$outcome)[1]),
      p = full_model$arma[1],
      d = full_model$arma[6],
      q = full_model$arma[2],
      arima_order = paste0("ARIMA(", full_model$arma[1], ",",
                           full_model$arma[6], ",",
                           full_model$arma[2], ")"),
      aic = round(AIC(full_model), 2),
      adf_p = round(adf$p.value, 4),
      lb_p = round(lb$p.value, 4),
      rmse = round(rmse, 4),
      mae = round(mae, 4),
      random_walk = ifelse(full_model$arma[6] >= 1 & lb$p.value > 0.05,
                           "Yes", "No"),
      stringsAsFactors = FALSE
    )
  }, error = function(e) NULL)
}

results <- map_dfr(eligible_tickers, fit_arima_ticker)
```

```
cat("Tickers successfully fit:", nrow(results), "\n")
```

```
## Tickers successfully fit: 347
```

```
cat("Random walk tickers:", sum(results$random_walk == "Yes"),
    "/", nrow(results), "\n")
```

```
## Random walk tickers: 244 / 347
```

Results Summary Table (Top 20 by Trade Count)

```
results_sorted <- arrange(results, desc(n_trades))
results_top20 <- head(results_sorted, 20)
results_display <- select(results_top20, ticker, n_trades, outcome,
                          arima_order, adf_p, lb_p, rmse, random_walk)
knitr::kable(results_display, digits = 4,
             caption = "ARIMA Results for Top 20 Most-Traded Tickers")
```

ARIMA Results for Top 20 Most-Traded Tickers

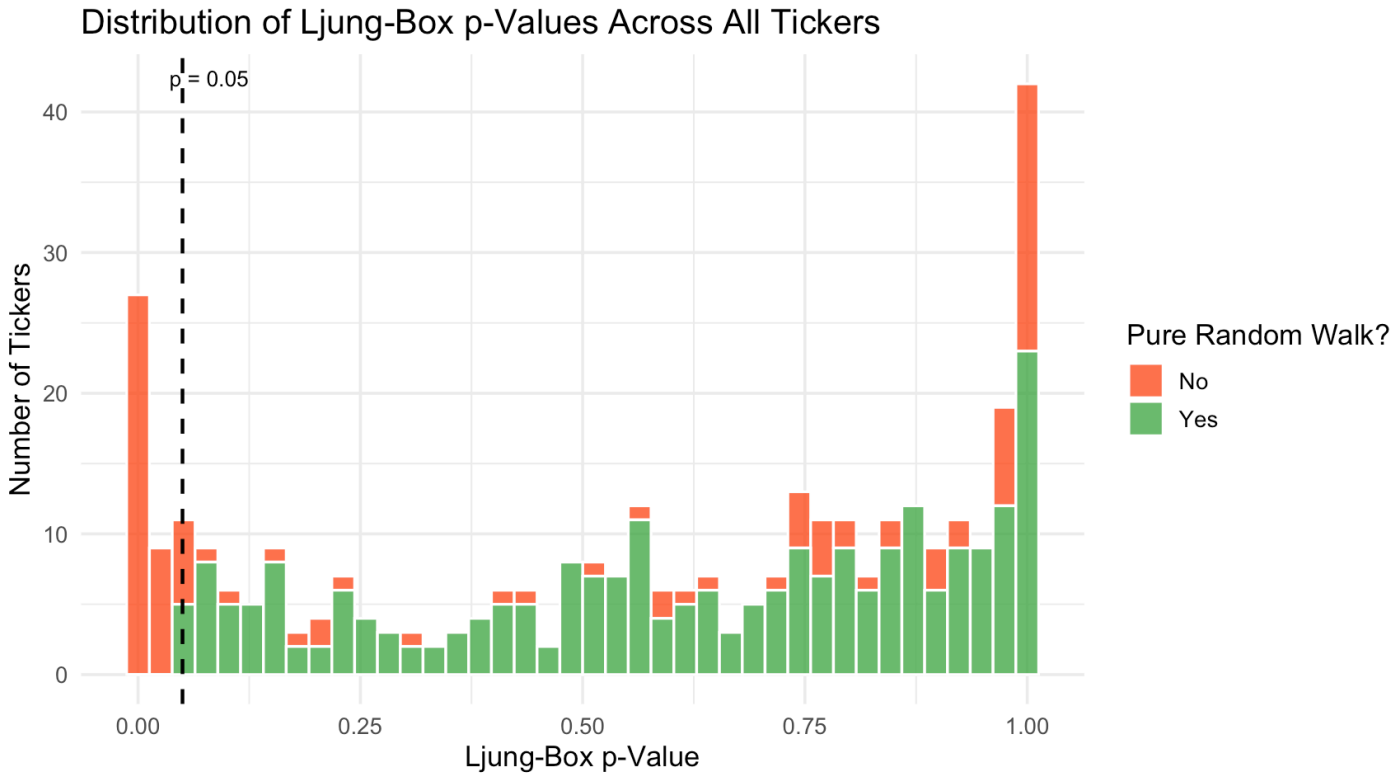
ticker	n_trades	outcome	arima_order	adf_p	lb_p	rmse	random_walk
KXUSAIRANAGREEMENT-27-26APR	8379	0	ARIMA(5,1,5)	0.0100	0.0000	0.0109	No
KXATPCHALLENGERMATCH-26APR02GUIHUE-HUE	8355	1	ARIMA(2,1,2)	0.2759	0.0002	0.0704	No
KXDHSFUNDING-26APR01	5280	0	ARIMA(1,1,2)	0.5825	0.0052	0.3163	No
KXATPCHALLENGERMATCH-26APR02GUIHUE-GUI	4776	0	ARIMA(5,1,1)	0.5601	0.0075	0.0929	No
KXATPCHALLENGERMATCH-26APR02HEIPRA-PRA	3554	0	ARIMA(5,1,2)	0.2823	0.3628	0.0964	Yes
KXNBASPREAD-26APR02SASLAC-SAS19	2928	0	ARIMA(5,1,3)	0.8401	0.0000	0.4134	No
KXNBASPREAD-26APR02CLEGSW-CLE10	2730	0	ARIMA(5,1,2)	0.8067	0.0291	0.1912	No
KXNBASPREAD-26APR02SASLAC-SAS10	2456	1	ARIMA(4,1,3)	0.8293	0.3336	0.0303	Yes
KXNBASPREAD-26APR02SASLAC-SAS4	2181	1	ARIMA(5,1,3)	0.7799	0.0002	0.2421	No
KXATPCHALLENGERMATCH-26APR02HEIPRA-HEI	2096	1	ARIMA(0,1,1)	0.1828	0.0667	0.0435	Yes

ticker	n_trades	outcome	arima_order	adf_p	lb_p	rmse	random_walk
KXNBATOTAL-26APR02CLEGSW-226	2079	1	ARIMA(5,1,2)	0.6547	0.2660	0.2642	Yes
KXNBASPREAD-26APR02SASLAC-SAS13	2047	1	ARIMA(3,1,3)	0.8626	0.2234	0.2906	Yes
KXNBASPREAD-26APR02SASLAC-SAS7	2016	1	ARIMA(5,1,2)	0.8147	0.8405	0.1703	Yes
KXNBATOTAL-26APR02CLEGSW-229	1956	0	ARIMA(0,1,5)	0.4077	0.0154	0.1646	No
KXNBASPREAD-26APR02SASLAC-SAS16	1902	1	ARIMA(1,1,2)	0.6632	0.2165	0.1835	Yes
KXLEAVESTARMER-26APR01	1837	0	ARIMA(5,1,5)	0.0434	0.4839	0.0397	Yes
KXNBASPREAD-26APR02NOPPOR-POR7	1811	1	ARIMA(1,1,1)	0.9862	0.0001	0.2673	No
KXNBASPREAD-26APR02CLEGSW-CLE7	1792	0	ARIMA(1,1,2)	0.0410	0.0000	0.1314	No
KXNBASPREAD-26APR02SASLAC-LAC2	1534	0	ARIMA(4,2,2)	0.5441	0.3989	0.2283	Yes
KXNBASPREAD-26APR02CLEGSW-CLE4	1442	1	ARIMA(0,1,2)	0.6592	0.0003	0.1127	No

Aggregate Visualizations

1. Ljung-Box p-Value Distribution

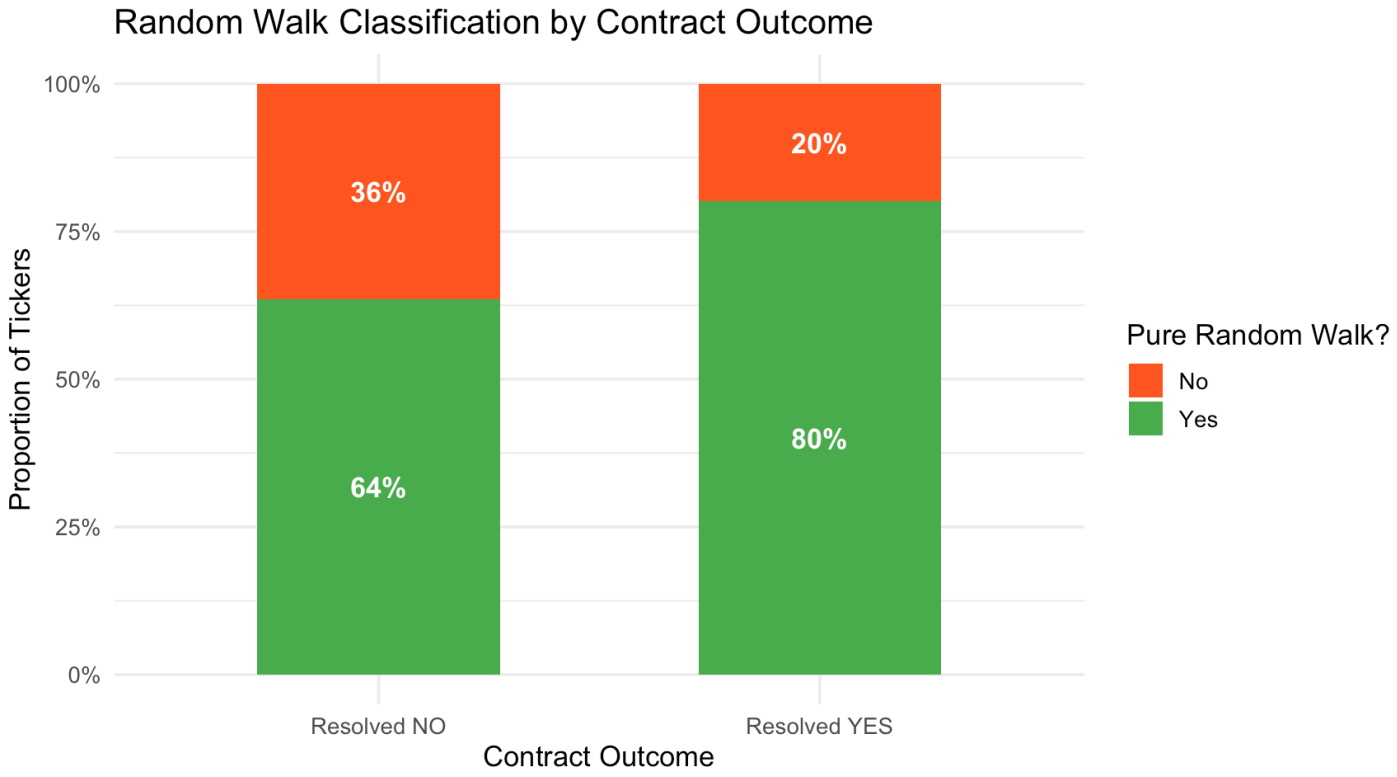
```
ggplot(results, aes(x = lb_p, fill = random_walk)) +
  geom_histogram(bins = 40, color = "white", alpha = 0.85) +
  geom_vline(xintercept = 0.05, linetype = "dashed",
             color = "black", linewidth = 0.8) +
  annotate("text", x = 0.08, y = Inf,
           label = "p = 0.05", vjust = 2, size = 3.5) +
  scale_fill_manual(values = c("Yes" = "#4CAF50", "No" = "#FF5722")) +
  labs(
    title = "Distribution of Ljung-Box p-Values Across All Tickers",
    x = "Ljung-Box p-Value",
    y = "Number of Tickers",
    fill = "Pure Random Walk?"
  ) +
  theme_minimal(base_size = 13)
```



2. Random Walk Classification by Outcome

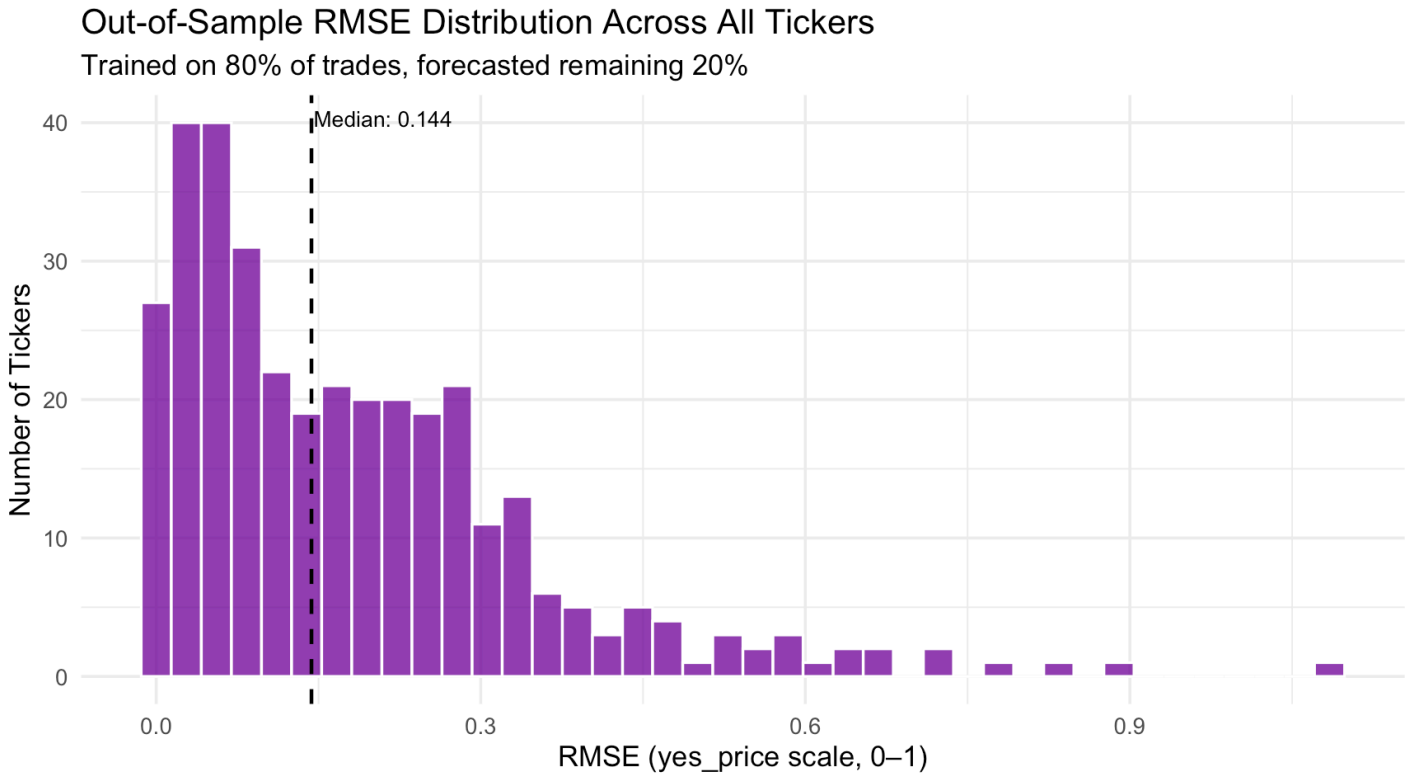
Do correctly-resolved contracts look more like random walks?

```
results_labeled <- mutate(results,
                          outcome_label = ifelse(outcome == 1,
                                                 "Resolved YES",
                                                 "Resolved NO"))
rw_outcome_counts <- count(results_labeled, outcome_label, random_walk)
rw_outcome_grp <- group_by(rw_outcome_counts, outcome_label)
rw_outcome <- mutate(rw_outcome_grp, pct = n / sum(n))

ggplot(rw_outcome, aes(x = outcome_label, y = pct, fill = random_walk)) +
  geom_col(position = "stack", width = 0.55) +
  geom_text(aes(label = scales::percent(pct, accuracy = 1)),
            position = position_stack(vjust = 0.5),
            color = "white", size = 4.5, fontface = "bold") +
  scale_fill_manual(values = c("Yes" = "#4CAF50", "No" = "#FF5722")) +
  scale_y_continuous(labels = scales::percent) +
  labs(
    title = "Random Walk Classification by Contract Outcome",
    x = "Contract Outcome",
    y = "Proportion of Tickers",
    fill = "Pure Random Walk?"
  ) +
  theme_minimal(base_size = 13)
```



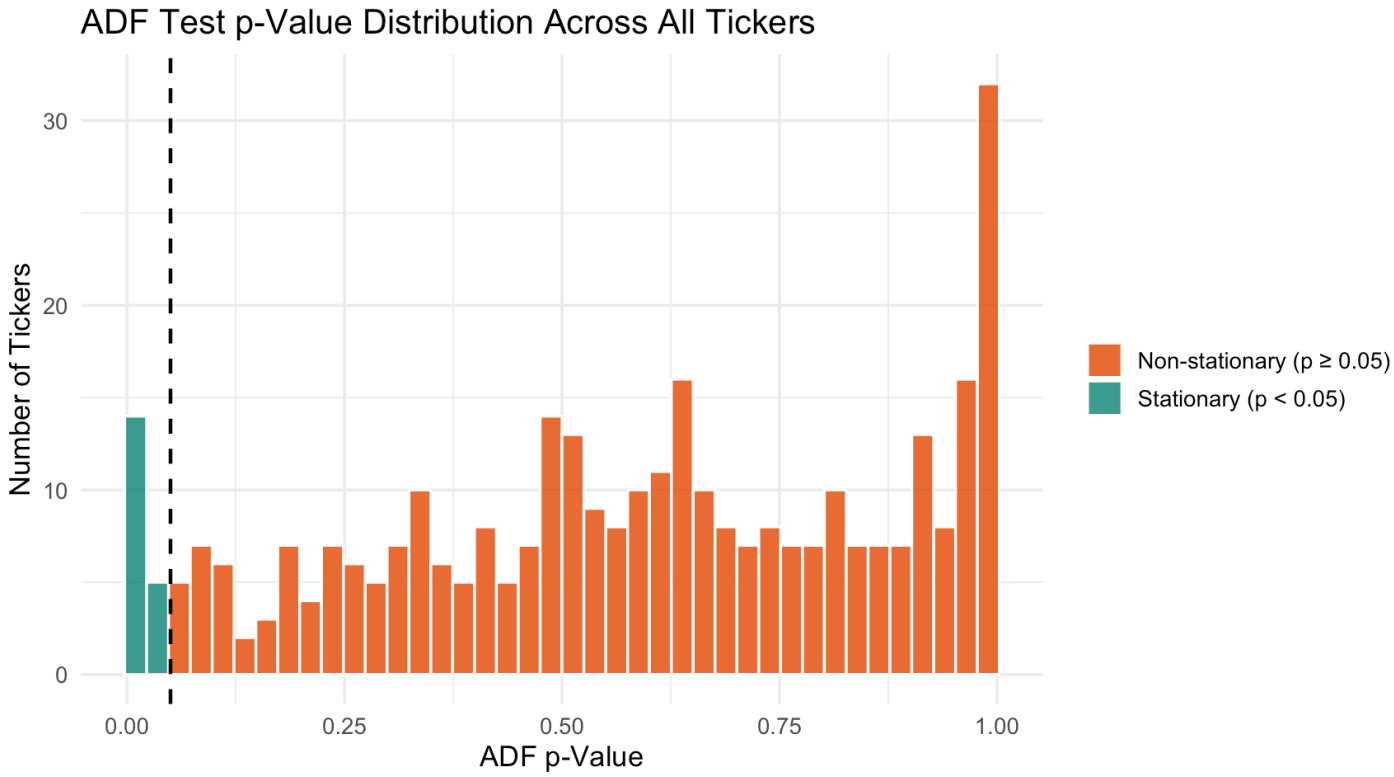
3. Out-of-Sample RMSE Distribution

```
median_rmse <- median(results$rmse, na.rm = TRUE)

ggplot(results, aes(x = rmse)) +
  geom_histogram(bins = 40, fill = "#7B1FA2", color = "white", alpha = 0.85) +
  geom_vline(xintercept = median_rmse,
             linetype = "dashed", color = "black", linewidth = 0.8) +
  annotate("text", x = median_rmse + 0.002, y = Inf,
           label = paste0("Median: ", round(median_rmse, 3)),
           vjust = 2, hjust = 0, size = 3.5) +
  labs(
    title = "Out-of-Sample RMSE Distribution Across All Tickers",
    subtitle = "Trained on 80% of trades, forecasted remaining 20%",
    x = "RMSE (yes_price scale, 0\u20131)",
    y = "Number of Tickers"
  ) +
  theme_minimal(base_size = 13)
```



4. ADF p-Value Distribution

```
adf_summary <- mutate(results,
                      stationary = ifelse(adf_p < 0.05,
                                          "Stationary (p < 0.05)",
                                          "Non-stationary (p \u2265 0.05)"))

ggplot(adf_summary, aes(x = adf_p, fill = stationary)) +
  geom_histogram(bins = 40, color = "white", alpha = 0.85) +
  geom_vline(xintercept = 0.05, linetype = "dashed",
             color = "black", linewidth = 0.8) +
  scale_fill_manual(values = c("Stationary (p < 0.05)" = "#00897B",
                               "Non-stationary (p \u2265 0.05)" = "#E65100")) +
  labs(
    title = "ADF Test p-Value Distribution Across All Tickers",
    x = "ADF p-Value",
    y = "Number of Tickers",
    fill = NULL
  ) +
  theme_minimal(base_size = 13)
```



5

```
results_by_outcome <- mutate(results,
                             outcome_label = ifelse(outcome == 1,
                                                    "Resolved YES",
                                                    "Resolved NO"))
results_grouped <- group_by(results_by_outcome, outcome_label)
outcome_summary <- summarise(results_grouped,
  n_tickers = n(),
  pct_random_walk = round(mean(random_walk == "Yes") * 100, 1),
  median_rmse = round(median(rmse, na.rm = TRUE), 4)
)
knitr::kable(outcome_summary, caption = "ARIMA Results by Contract Outcome")
```

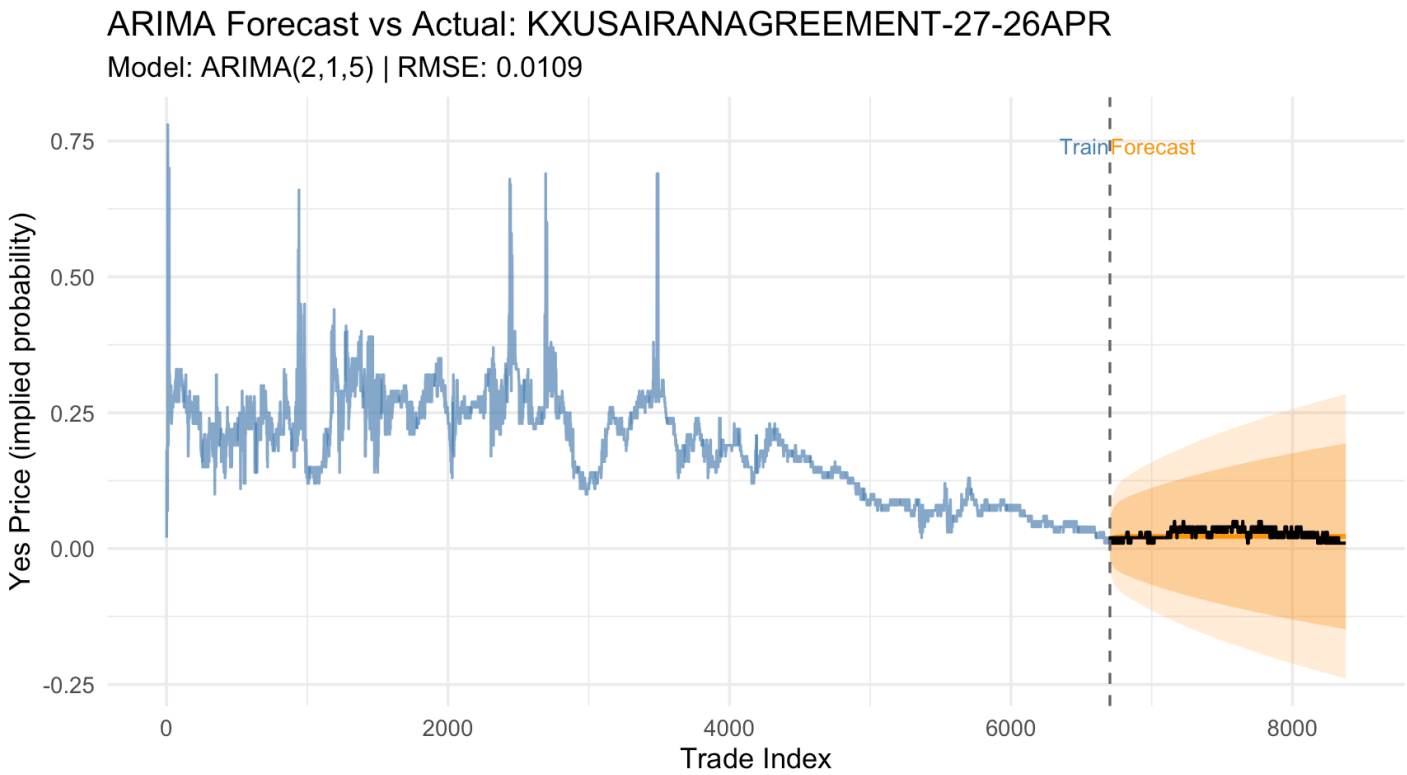
ARIMA Results by Contract Outcome

outcome_label	n_tickers	pct_random_walk	median_rmse
Resolved NO	206	63.6	0.1301
Resolved YES	141	80.1	0.1682

Forecast: Top Contract

```
ex_ticker <- trade_counts$ticker[1]
ex_df_raw <- filter(trades, ticker == ex_ticker)
ex_df <- arrange(ex_df_raw, created_time)
n_ex <- nrow(ex_df)
n_train <- floor(n_ex * 0.8)

train_ts <- ts(ex_df$yes_price[1:n_train])
ex_model <- auto.arima(train_ts, stepwise = TRUE, approximation = TRUE)
n_test <- n_ex - n_train
ex_fcast <- forecast(ex_model, h = n_test)

fcast_df <- data.frame(
  idx = (n_train + 1):n_ex,
  actual = ex_df$yes_price[(n_train + 1):n_ex],
  forecast = as.numeric(ex_fcast$mean),
  lo80 = as.numeric(ex_fcast$lower[, 1]),
  hi80 = as.numeric(ex_fcast$upper[, 1]),
  lo95 = as.numeric(ex_fcast$lower[, 2]),
  hi95 = as.numeric(ex_fcast$upper[, 2])
)

train_df <- data.frame(idx = 1:n_train, actual = ex_df$yes_price[1:n_train])

forecast_rmse <- round(sqrt(mean((fcast_df$forecast - fcast_df$actual)^2)), 4)
model_label <- paste0("ARIMA(", ex_model$arma[1], ",",
                      ex_model$arma[6], ",", ex_model$arma[2], ")")

ggplot() +
  geom_line(data = train_df,
            aes(x = idx, y = actual),
            color = "steelblue", alpha = 0.7) +
  geom_ribbon(data = fcast_df, aes(x = idx, ymin = lo95, ymax = hi95),
              fill = "#FF9800", alpha = 0.2) +
  geom_ribbon(data = fcast_df, aes(x = idx, ymin = lo80, ymax = hi80),
              fill = "#FF9800", alpha = 0.35) +
  geom_line(data = fcast_df,
            aes(x = idx, y = forecast),
            color = "#FF9800", linewidth = 1) +
  geom_line(data = fcast_df,
            aes(x = idx, y = actual),
            color = "black", linewidth = 0.7) +
  geom_vline(xintercept = n_train, linetype = "dashed", color = "gray40") +
  annotate("text", x = n_train - 5, y = max(ex_df$yes_price) * 0.95,
           label = "Train", hjust = 1, size = 3.5, color = "steelblue") +
  annotate("text", x = n_train + 5, y = max(ex_df$yes_price) * 0.95,
           label = "Forecast", hjust = 0, size = 3.5, color = "#FF9800") +
  labs(
    title = paste("ARIMA Forecast vs Actual:", substr(ex_ticker, 1, 45)),
    subtitle = paste0("Model: ", model_label, " | RMSE: ", forecast_rmse),
    x = "Trade Index",
```

```
    y = "Yes Price (implied probability)"
  ) +
  theme_minimal(base_size = 13)
```



Save Results

```
write_csv(results, "output/arima_results_all_tickers.csv")
```